

[Marcar como hecha](#)

12.1. Características de la tecnología JSP y componentes

La Web dinámica se ha desarrollado desde un sistema de información distribuido hypermedia (HTML) basado en red que ofrecía información estática hasta un conjunto de portales y aplicaciones en Internet que ofrecen un conjunto variado de servicios.

Las soluciones de primera generación incluyeron CGI, que es un mecanismo para ejecutar programas externos en un servidor web. El problema con los scripts CGI es la escalabilidad; se crea un nuevo proceso para cada petición.

Las soluciones de segunda generación incluyeron vendedores de servidores Web que proporcionaban plug-ins y APIs para sus servidores. El problema es que sus soluciones eran específicas a sus productos servidores. Microsoft proporcionó las páginas activas del servidor (ASP) que hicieron más fácil crear el contenido dinámico. Sin embargo, su solución sólo trabajaba con Microsoft IIS o Personal Web Server. Otra tecnología de segunda generación son los Servlets. Los Servlets hacen más fácil escribir aplicaciones del lado del servidor usando la tecnología Java. El problema con los CGI o los Servlets, sin embargo, es que tenemos que seguir el ciclo de vida de escribir, compilar y desplegar .

Las páginas JSP son una solución de tercera generación que se pueden combinar fácilmente con algunas soluciones de la segunda generación, creando el contenido dinámico, y haciendo más fácil y más rápido construir las aplicaciones basadas en Web que trabajan con una variedad de otras tecnologías: servidores Web, navegadores Web, servidores de aplicación y otras herramientas de desarrollo.

La tecnología Java Server Pages™ (JSP) nos permite poner segmentos de código servlet directamente dentro de una página HTML estática. Cuando el navegador carga una página JSP, se ejecuta el código del servlet y el servidor de aplicaciones crea, compila, carga y ejecuta un servlet en segundo plano para ejecutar los segmentos de código servlet y devolver una página HTML o imprimir un informe XML.

Modelo cliente-servidor

Se puede hablar de lenguajes de lado servidor, por un lado, de aquellos lenguajes que son reconocidos, ejecutados e interpretados por el propio servidor y que se envían al cliente en un formato comprensible para él. Por otro lado, los lenguajes de lado cliente (entre los cuales no sólo se encuentra el HTML sino también el Java y el JavaScript los cuales son simplemente incluidos en el código HTML) son aquellos que pueden ser directamente "digeridos" por el navegador y no necesitan un pretratamiento.

Los programadores de aplicaciones JAVA mediante la especificación J2EE escriben componentes de aplicación J2EE. Un componente J2EE es una unidad de software funcional auto-contenida que se ensambla dentro de una aplicación J2EE y que se comunica con otros componentes de aplicación. La especificación J2EE define los siguientes componentes de aplicación:

- Componentes de Aplicación Cliente: Navegador y páginas jsp renderizadas
- Componentes JavaBeans Enterprise: clases de negocio (EJBs)
- Componentes Servlets y JavaServer Pages (también llamados componentes Web): clases controladoras.
- Applets: pequeñas aplicaciones que se ejecutan en el cliente

Formas de seguir la trayectoria de los usuarios (repaso)

Los servlets permiten seguir la trayectoria de un cliente, es decir, obtener y mantener una determinada información acerca del cliente. De esta forma se puede tener identificado a un cliente (usuario que está utilizando un browser) durante un determinado tiempo. Esto es muy importante si se quiere disponer de aplicaciones que impliquen la ejecución de varios servlets o la ejecución repetida de un mismo servlet. Un claro ejemplo de aplicación de esta técnica es el de los comercios vía Internet que permiten llevar un carrito de la compra en el que se van

?

guardando aquellos productos solicitados por el cliente. El cliente puede ir navegando por las distintas secciones del comercio virtual, es decir realizando distintas conexiones HTTP y ejecutando diversos servlets, y a pesar de ello no se pierde la información contenida en el carrito de la compra y se sabe en todo momento que es un mismo cliente quien está haciendo esas conexiones diferentes.

El mantener información sobre un cliente a lo largo de un proceso que implica múltiples conexiones se puede realizar de tres formas distintas:

1. Mediante cookies
2. Mediante seguimiento de sesiones (Session Tracking)
3. Mediante la reescritura de URLs y paso de parámetros en formulario (Request)

Introducción a JSP

La tecnología JSP (Java Server Pages) es una especificación abierta desarrollada por Sun Microsystems como una alternativa a Active Server Pages (ASP) de Microsoft, y son un componente dominante de la especificación de Java 2 Enterprise Edition (J2EE). Muchos de los servidores de aplicaciones comercialmente disponibles (como BEA WebLogic, IBM WebSphere, Live JRun, Orion, etcétera) ya utilizan tecnología JSP.

Esta tecnología permite desarrollar páginas web con contenido dinámico y supone una evolución frente a la tecnología CGI, y los Servlets, a los que complementa.

Un fichero JSP puede contener etiquetas HTML normales, y elementos especiales para generar el contenido dinámico.

Al mismo tiempo permite una separación en n capas de la arquitectura de la aplicación web y se integra perfectamente con todas las API's empresariales de Java: JDBC, RMI (y CORBA), JNDI, EJB, JMS, JTA, ...

Estructura de una página JSP

Una página JSP es básicamente una página Web con HTML tradicional y código Java. La extensión de fichero de una página JSP es ".jsp" en vez de ".html" o ".htm", y eso le dice al servidor que esta página requiere un manejo especial que se conseguirá con una extensión del servidor o un plug-in.

Un ejemplo sencillo:

```
<%@ page language="java" contentType="text/html" %>
<html>
  <head>
    <title>Hola, mundo!!</title>
  </head>
  <body>
    <h1>Hola, mundo!</h1>
    Hoy es <%= new java.util.Date() %>.
  </body>
</html>
```

Directivas

Las directivas JSP son instrucciones procesadas por el motor JSP cuando la página JSP se traduce a un servlet. Las directivas usadas en este ejemplo le dicen al motor JSP que incluya ciertos paquetes y clases. Las directivas están encerradas entre etiquetas de directiva <%@ y %>.

```
<%@ page import="javax.naming.*" %>
<%@ page import="javax.rmi.PortableRemoteObject" %>
<%@ page import="Beans.*" %>
```

Declaraciones

Las declaraciones JSP nos permiten configurar variables para su uso posterior en expresiones o scriptlets. También podemos declarar variables dentro de expresiones o scriptlets en el momento de usarlas. El ámbito es toda la página JSP, no hay concepto de variables de ejemplar. Es decir, no tenemos que declarar variables de ejemplar para usar en más de una expresión o scriptlet. Las declaraciones van encerradas entre etiquetas de declaración <%! y %>.

?

Podemos tener varias declaraciones. Por ejemplo,

```
<%! double bonus; String text; %> .
<%! String strMult, socsec; %>
<%! Integer integerMult; %>
<%! int multiplier; %>
<%! double bonus; %>
```

Scriptlets

Los scriptlets JSP nos permiten embeber segmentos de código java dentro de una página JSP. El código embebido se inserta directamente en el servlet generado que se ejecuta cuando se pide la página. Este scriptlet usa las variables declaradas en las directivas descritas arriba. Los Scriptlets van encerrados entre etiquetas `<%` y `%>`.

```
<%
strMult = request.getParameter("MULTIPLIER");
socsec = request.getParameter("SOCSEC");
integerMult = new Integer(strMult);
multiplier = integerMult.intValue();
bonus = 100.00;
%>
```

Variables Predefinidas

Un scriptlet puede usar las siguientes variables predefinidas: **session**, **request**, **response**, **out**, **e in**. Este ejemplo usa la variable predefinida `request`, que es un objeto `HttpServletRequest`. De igual forma, `response` es un objeto `HttpServletResponse`, `out` es un objeto `PrintWriter`, `e in` es un objeto `BufferedReader`. Las variables predefinidas se usan en los scriptlets de la misma forma que se usan en los servlets, excepto que no las declaramos. El código es el mismo que el anterior

```
<%
strMult = request.getParameter("MULTIPLIER");
socsec = request.getParameter("SOCSEC");
integerMult = new Integer(strMult);
multiplier = integerMult.intValue();
bonus = 100.00;
%>
```

Expresiones

Las expresiones JSP nos permiten recuperar dinámicamente o calcular valores a insertar directamente en la página JSP. En este ejemplo, una expresión recupera el número de la seguridad social desde el bean de entidad `Bonus` y lo pone en la página JSP.

```
<H1>Bonus Calculation</H1>
Social security number retrieved:
<%= record.getSocSec() %>
<P>
Bonus Amount retrieved: <%= record.getBonus() %>
<P>
```

Etiquetas específicas de JSP

La especificación JavaServer Pages 1.1 define etiquetas específicas de JSP que nos permiten extender la implementación JSP con nuevas características y ocultar mucha complejidad a los diseñadores visuales que necesitan buscar la página JSP y modificarla

